AIML CZG565 · ML COMPANION READER

Linear Models for Classification

Logistic regression, confusion matrix, ROC, and multiclass strategies

Module 4 · Read with slides ML CS-5 and CS-6 · Every key example worked out

How to use this companion. Blue boxes give intuition first. Amber boxes give a relatable analogy. Green boxes solve examples with all steps. Red boxes show common mistakes. Purple boxes give one-line recall points.

1 Classification vs Regression

In regression, the output is a real number. In classification, the output is a class label. For binary classification, the output is usually 0 or 1. For example, "job offered" can be 1 and "not offered" can be 0.

☞ The intuition

Classification answers "which bucket?" not "how much?". The model must separate classes with a boundary. A linear classifier uses a line in 2-D and a plane in 3-D.

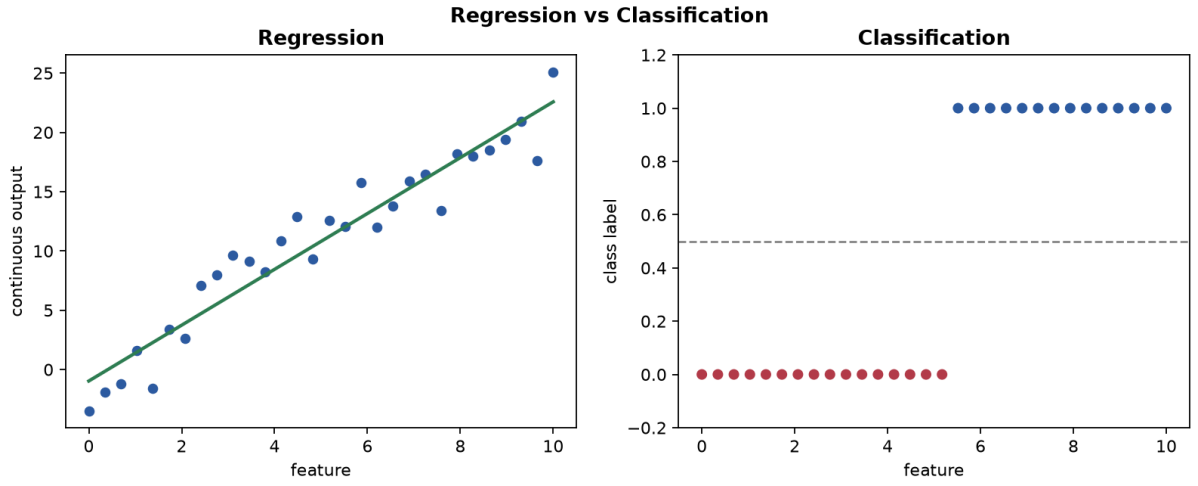

Figure 1: Regression predicts a continuous value. Classification predicts one of discrete classes. The decision boundary splits the feature space.

★ Everyday picture

Think of a college admission desk. The candidate goes to one of two trays: selected or not selected. This is a binary classification decision.

✓ **Key takeaway**

Regression predicts numbers. Classification predicts labels. Binary classification is the foundation for logistic regression.

2 Why Not Use Linear Regression for Classification?

A linear model $h(x) = w_0 + w_1 x$ can output any real number. For classification, we need outputs in $[0, 1]$ interpreted as probability. Linear regression can give $h(x) > 1$ or $h(x) < 0$, which is invalid as probability.

☞ The intuition

Using linear regression for classification is like a thermometer with no limits. It can go below 0% chance and above 100% chance. That breaks probabilistic interpretation.

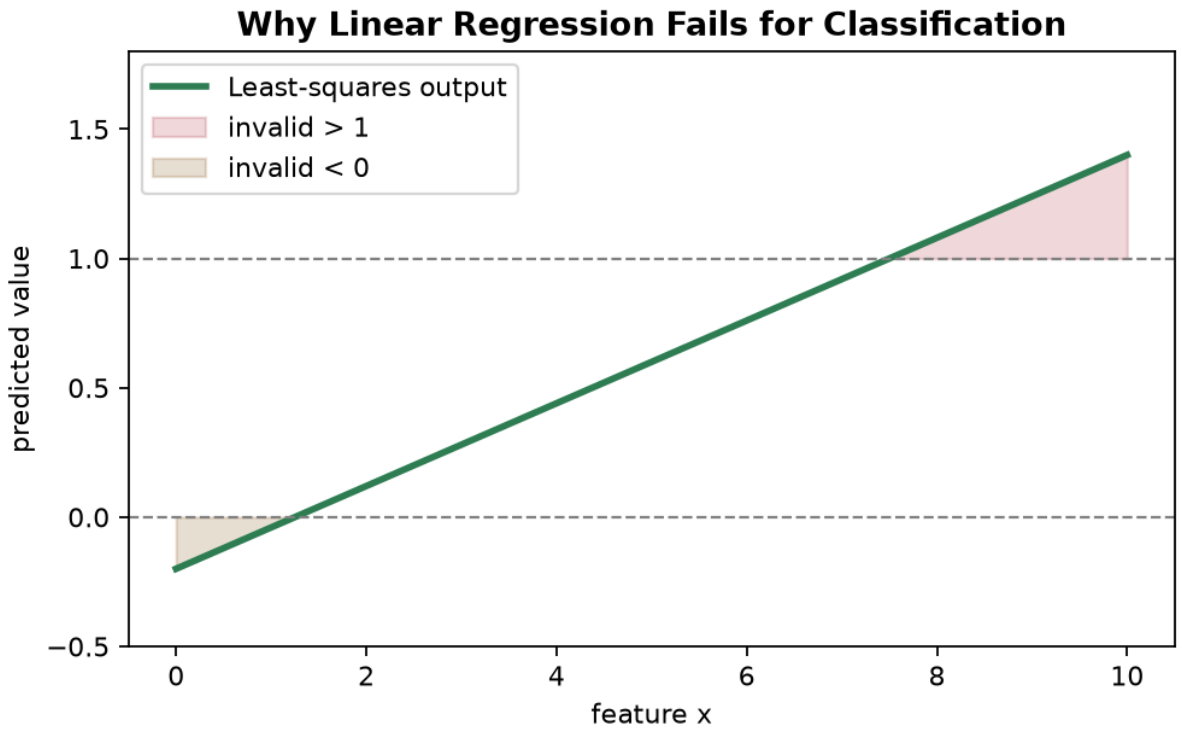

Figure 2: A linear fit for binary labels can produce values below 0 and above 1. These cannot be valid probabilities.

✗ Watch out

Linear regression is also sensitive to outliers in a way that can move the class boundary badly. One extra point far away can change the decision line a lot.

✓ Key takeaway

For classification, we need bounded outputs and a better loss than least squares. Logistic regression solves both issues.

3 Logistic Regression and the Sigmoid Function

Logistic regression first builds a linear score:

$$z = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d.$$

Then it maps this score through the sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

The output $\hat{y} = \sigma(z)$ is in $(0, 1)$.

☞ The intuition

The sigmoid is an S-shaped squash. Big negative scores map near 0. Big positive scores map near 1. A score of 0 maps to 0.5.

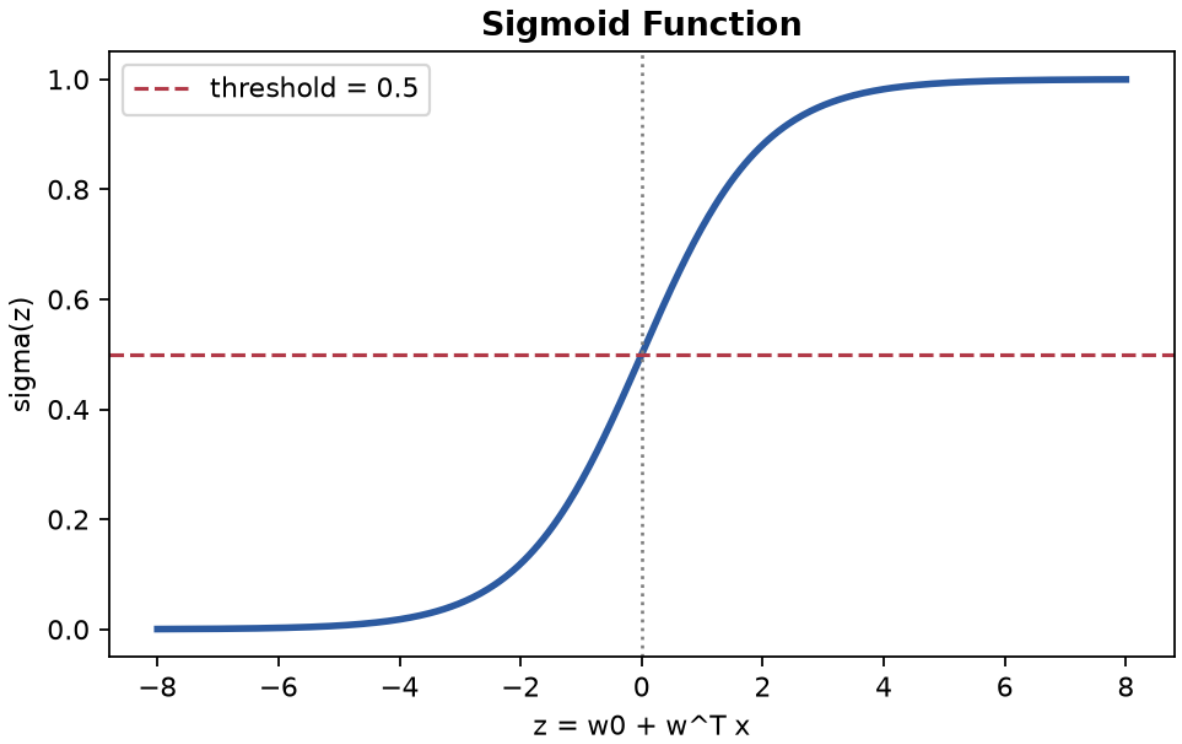

Figure 3: The sigmoid curve maps any real score to a probability between 0 and 1. The midpoint is at $z = 0$ where probability is 0.5.

✎ Worked example: Prediction with a logistic model

Assume model:

$$z = 0.4 + 0.3\,\text{CGPA} - 0.45\,\text{IQ}.$$

For candidate $(\text{CGPA} = 5, \text{IQ} = 6)$:

Step 1. Compute score:

$$z = 0.4 + 0.3(5) - 0.45(6) = 0.4 + 1.5 - 2.7 = -0.8.$$

Step 2. Compute probability:

$$\hat{y} = \sigma(-0.8) = \frac{1}{1 + e^{0.8}} \approx 0.31.$$

Step 3. Apply threshold 0.5: since $0.31 < 0.5$, predict class 0.

✓ **Key takeaway**

Logistic regression = linear score + sigmoid. It gives probabilities that can be thresholded for class prediction.

4 Decision Boundary: Linear and Nonlinear

With threshold 0.5, prediction is 1 when $\sigma(z) \geq 0.5$. Since $\sigma(z) \geq 0.5$ exactly when $z \geq 0$, decision boundary is:

$$w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d = 0.$$

This is linear in original features.

If we engineer nonlinear features (e.g. x_1^2, $x_1 x_2$), the boundary can be nonlinear in original space while model is still linear in parameters.

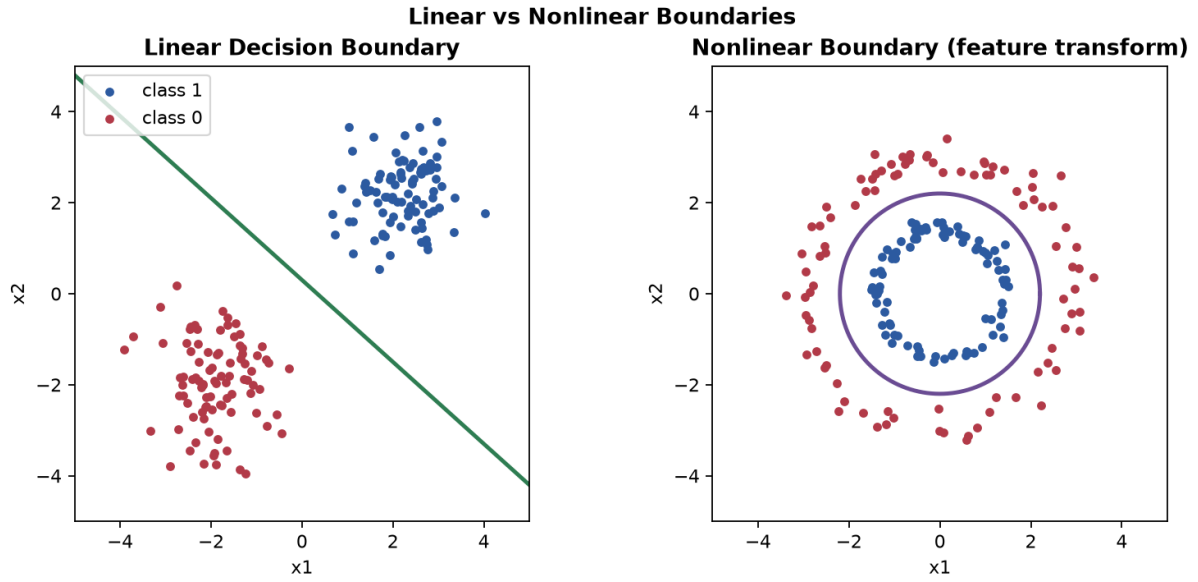

Figure 4: Left: linear decision boundary. Right: nonlinear boundary achieved by adding engineered polynomial features.

★ Everyday picture

A simple pass/fail rule can be one straight cut-off line. A richer policy can use curved zones made from multiple conditions. Feature engineering creates those curved zones.

✓ Key takeaway

Logistic regression has a linear boundary in feature space. Add transformed features to model nonlinear boundaries.

5 Cost Function: Cross-Entropy

Least-squares cost is not ideal for logistic regression. We use cross-entropy (log loss):

$$J(\mathbf{w}) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\hat{y}^{(i)} + (1-y^{(i)})\log(1-\hat{y}^{(i)})\right].$$

For one sample:

$$\text{Cost}(\hat{y}, y) = \begin{cases} -\log(\hat{y}) & y = 1, \\ -\log(1-\hat{y}) & y = 0. \end{cases}$$

☞ The intuition

If true label is 1, then predicting 0.99 should have low loss. Predicting 0.01 should have huge loss. Cross-entropy does exactly this.

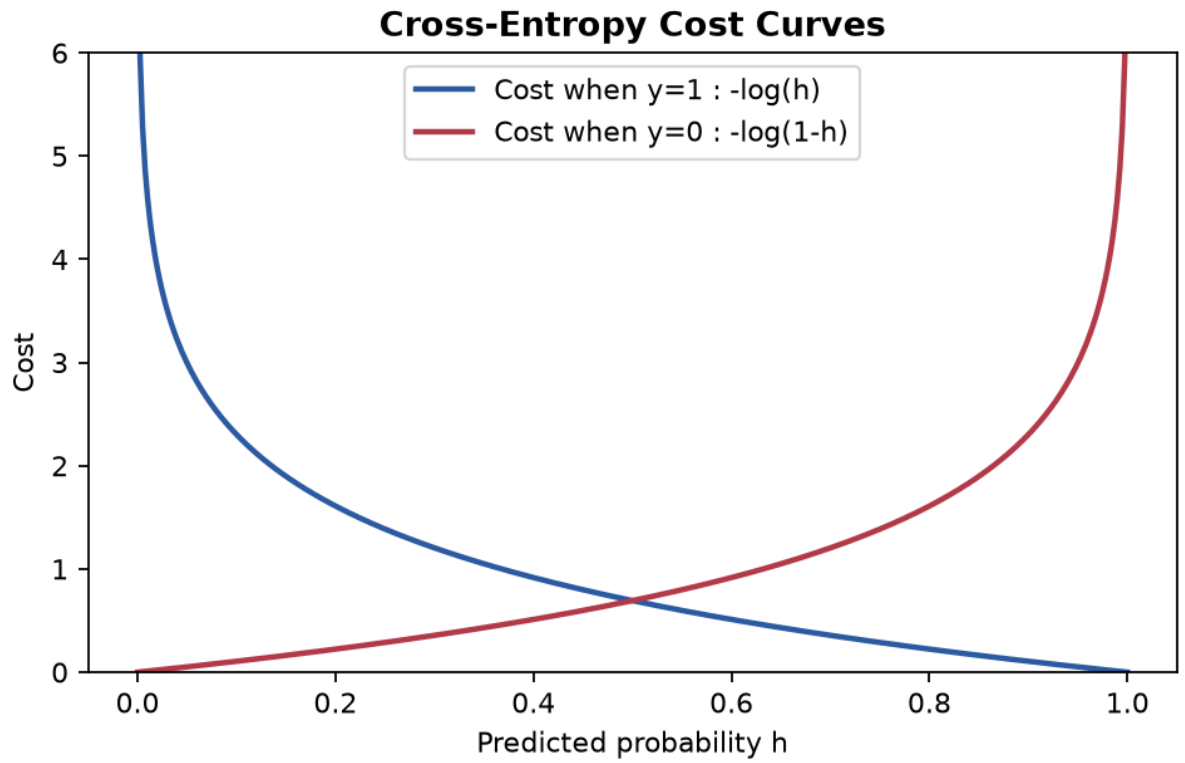

Figure 5: Cross-entropy penalty curves: for $y = 1$, loss explodes when predicted probability goes to 0; for $y = 0$, loss explodes when probability goes to 1.

✗ Watch out

Never use hard 0 or 1 probabilities in logs. Implementations clip predictions slightly to avoid $\log(0)$.

✓ Key takeaway

Cross-entropy is convex for logistic regression and gives stable training behavior. It strongly penalizes confident wrong predictions.

6 Learning Parameters with Gradient Descent

For logistic regression, gradient descent update is:

$$w_j \leftarrow w_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}.$$

This looks similar to linear regression, but $\hat{y}^{(i)}$ now comes from sigmoid.

☞ The intuition

Each update moves weights so predicted probabilities get closer to true labels. If model predicts too high, update pushes score down. If model predicts too low, update pushes score up.

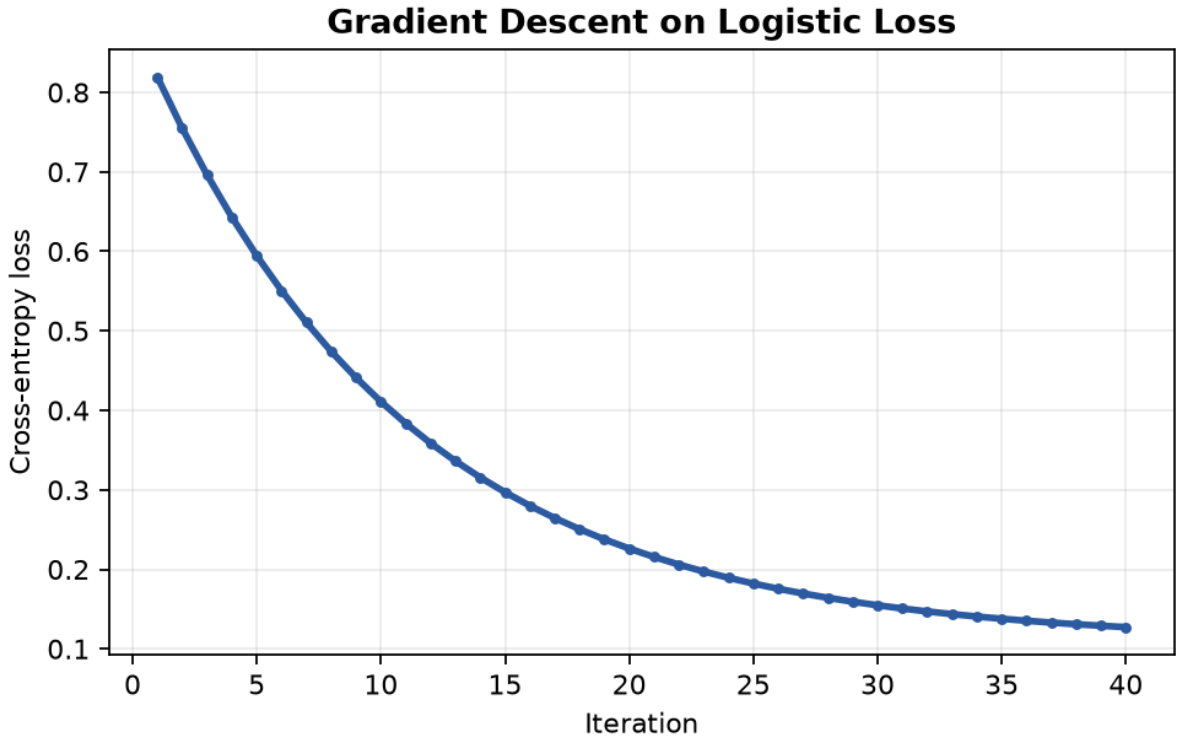

Figure 6: Gradient descent trajectory on logistic cost. The loss decreases as iterations proceed when learning rate is chosen well.

✎ Worked example: One mini gradient step (binary data)

Given one sample: $x = [1, 2]$, true label $y = 1$, current weights $w = [0.1, -0.2]$, learning rate $\alpha = 0.3$.

Step 1. Score: $z = 0.1(1) + (-0.2)(2) = -0.3$.

Step 2. Probability: $\hat{y} = \sigma(-0.3) \approx 0.426$.

Step 3. Error term: $(\hat{y} - y) = 0.426 - 1 = -0.574$.

Step 4. Gradient: $\nabla_w = (\hat{y} - y)x = [-0.574, -1.148]$.

Step 5. Update:

$$w \leftarrow w - \alpha \nabla_w = [0.1, -0.2] - 0.3[-0.574, -1.148] = [0.272, 0.144].$$

✓ **Key takeaway**

Gradient descent for logistic regression uses the same prediction-error structure with sigmoid probabilities. Learning rate controls step size exactly as before.

7 Regularization and Overfitting in Logistic Regression

Regularized logistic objective:

$$J_{\text{reg}}(\mathbf{w}) = J(\mathbf{w}) + \frac{\lambda}{2m}\sum_{j=1}^{d} w_j^2.$$

We usually do not regularize intercept w_0.

☞ The intuition

Regularization says: fit the data, but avoid very large weights. This reduces overfitting and improves generalization.

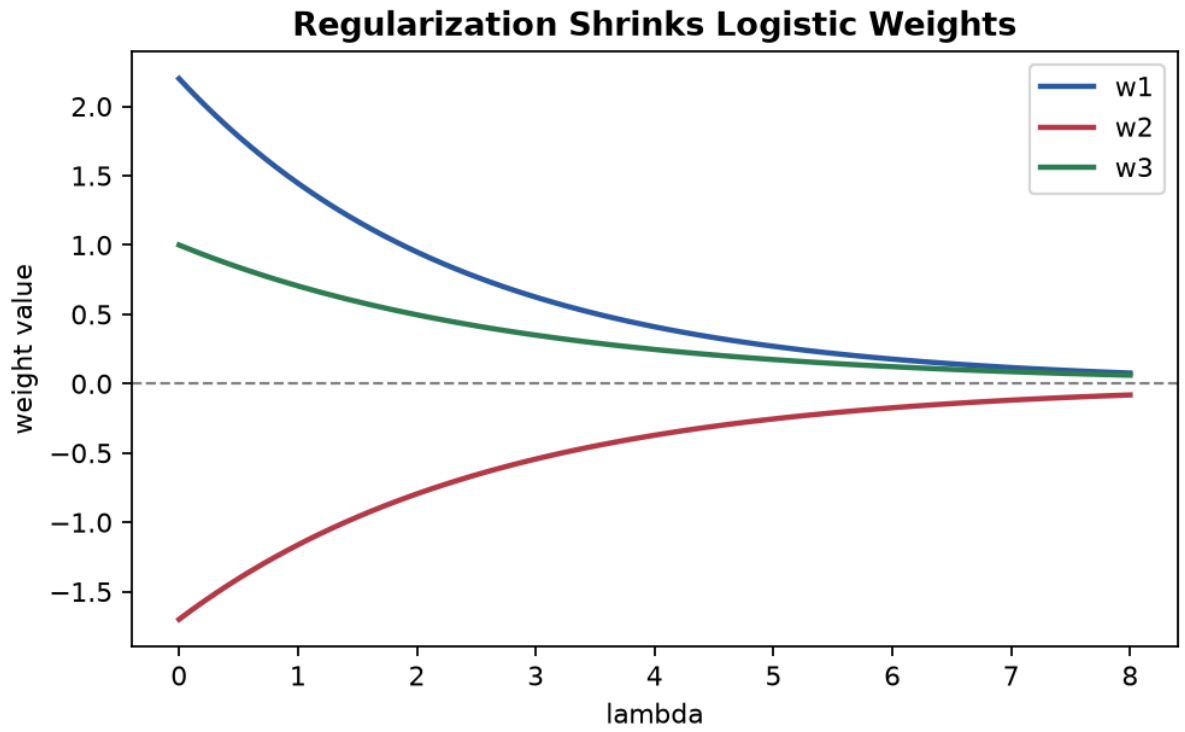

Figure 7: As regularization strength grows, weights shrink and boundaries become smoother, reducing variance.

✗ Watch out

If λ is too large, model underfits. If λ is too small, model can overfit. Tune it on validation data.

✓ Key takeaway

Regularization in logistic regression is conceptually same as module 3. Only the data-fit term is cross-entropy instead of MSE.

8 Confusion Matrix and Basic Metrics

For binary classes, confusion matrix has:

- TP: true positives,
- TN: true negatives,
- FP: false positives,
- FN: false negatives.

From these we compute:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}.$$

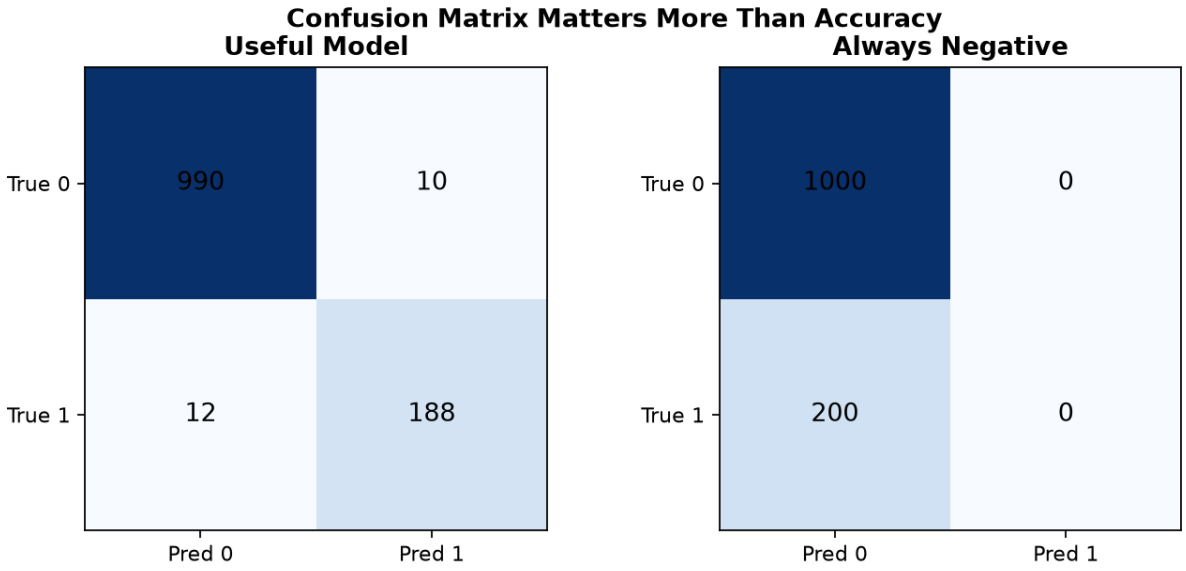

Figure 8: Two classifiers can have similar accuracy but very different error structure. Always inspect confusion matrix, not accuracy alone.

✎ Worked example: Metric computation from a confusion matrix

Suppose:

$$TP = 45, \quad FN = 5, \quad FP = 15, \quad TN = 135.$$

Step 1. Total samples: 200.

Step 2. Accuracy: $(45 + 135)/200 = 0.90$.

Step 3. Precision: $45/(45 + 15) = 0.75$.

Step 4. Recall: $45/(45 + 5) = 0.90$.

Step 5. F1-score:

$$F1 = \frac{2 \cdot 0.75 \cdot 0.90}{0.75 + 0.90} = 0.818.$$

✗ Watch out

High accuracy can be misleading on imbalanced data. Always include precision, recall, and F1.

✓ Key takeaway

Accuracy alone is not enough. Confusion matrix reveals what kind of mistakes model makes.

9 ROC Curve and AUC

ROC plots True Positive Rate (TPR) vs False Positive Rate (FPR) across thresholds.

$$\text{TPR} = \frac{TP}{TP + FN}, \qquad \text{FPR} = \frac{FP}{FP + TN}.$$

AUC (Area Under Curve) summarizes ROC into one number. Higher AUC means better ranking ability.

☞ The intuition

Threshold is a slider. Moving slider changes how aggressive classifier is. ROC shows all possible trade-offs between catching positives and raising false alarms.

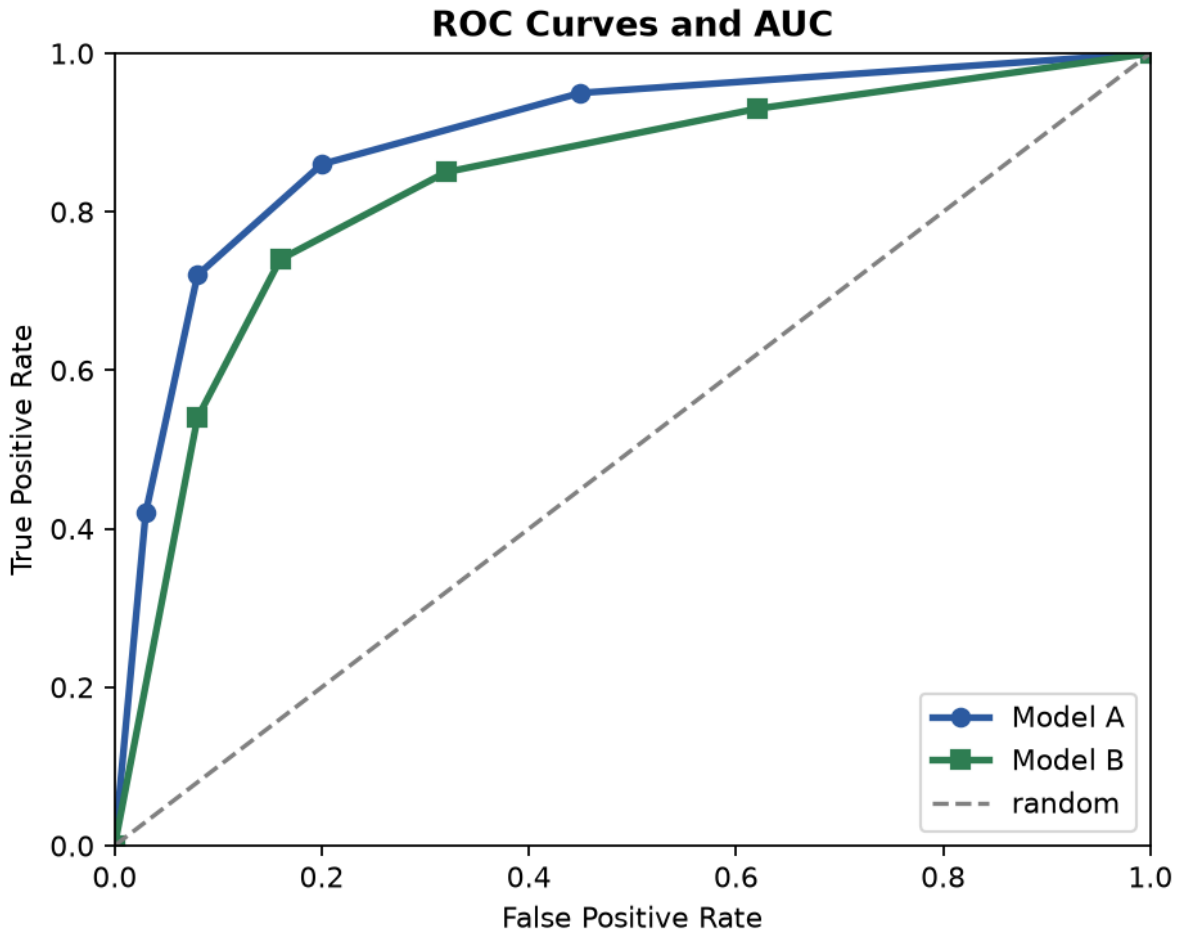

Figure 9: ROC curves for two models. Curve closer to top-left is better. AUC summarizes the full curve independent of one fixed threshold.

✎ Worked example: Construct first ROC points from ranked scores

Scores with true labels (sorted high to low): $(0.95, +), (0.90, +), (0.87, -), (0.60, +), (0.40, -)$. Total positives $P = 3$, negatives $N = 2$.

Step 1. Start threshold above 0.95: predict none positive. Point $(\text{FPR}, \text{TPR}) = (0, 0)$.

Step 2. Threshold at 0.95: one positive captured. $TPR = 1/3$, $FPR = 0/2 = 0$.

Step 3. Threshold at 0.90: two positives captured. $TPR = 2/3$, $FPR = 0$.

Step 4. Threshold at 0.87: one negative also predicted positive. $TPR = 2/3$, $FPR = 1/2$.

Step 5. Continue until all points predicted positive: end at $(1, 1)$.

✓ **Key takeaway**

ROC compares models over all thresholds. AUC is threshold-independent summary quality. Choose operating threshold based on business cost of FP vs FN.

10 Class Imbalance and Remedies

In many applications, positive class is rare. Example: disease positives may be 1% of all cases. A classifier predicting always "negative" gets 99% accuracy but is useless.

☞ The intuition

Imbalanced data is like searching for a few needles in a huge haystack. If you always say "no needle", you look accurate but fail the true goal.

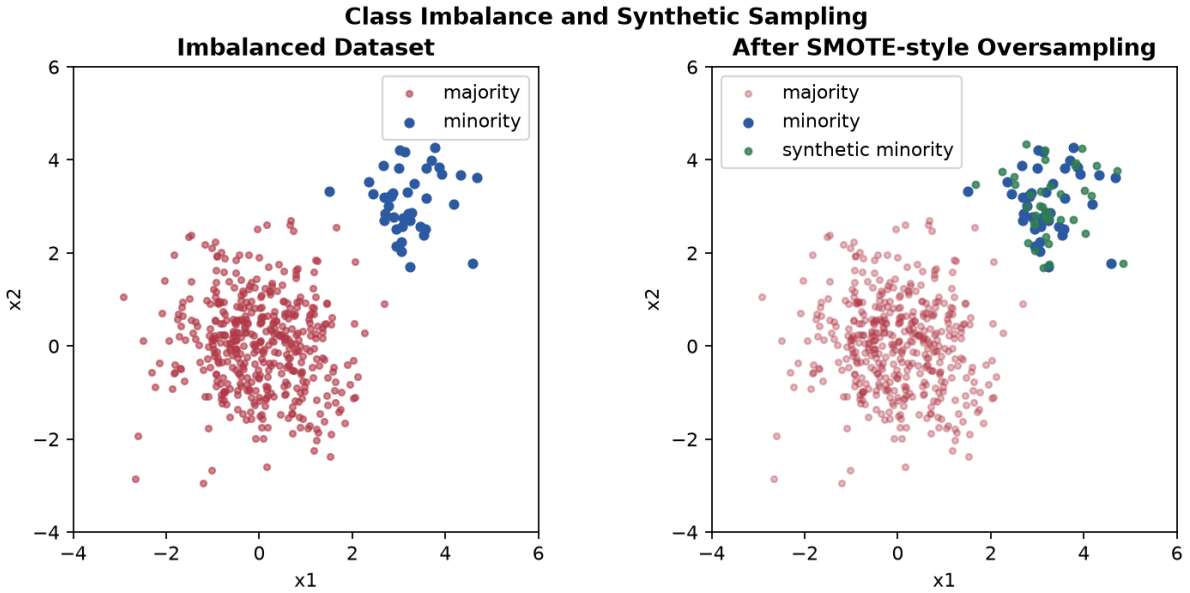

Figure 10: Severe class imbalance can hide poor minority-class performance. Resampling and class weights improve minority detection.

Common remedies:

- class-weighted loss,
- oversampling minority,
- synthetic oversampling (SMOTE),
- threshold tuning for recall/precision needs.

✗ Watch out

Never oversample before train-test split. Do split first, then oversample only training data to avoid data leakage.

✓ Key takeaway

For imbalanced tasks, optimize recall/precision trade-off, not raw accuracy. Use proper sampling and threshold strategy.

11 Multiclass Classification: One-vs-Rest and One-vs-One

Logistic regression is naturally binary. For K classes, common strategies are:

- One-vs-Rest (OvR): train K binary models.
- One-vs-One (OvO): train $K(K-1)/2$ binary models.

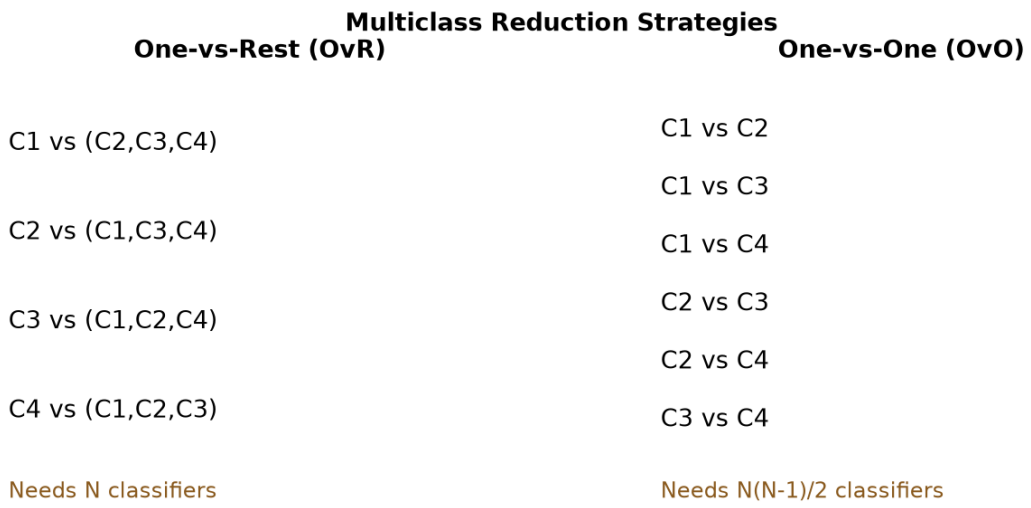

Figure 11: OvR trains one classifier per class against all others. OvO trains one classifier for each pair of classes and uses voting.

★ Everyday picture

OvR is like asking each class "Are you me or not?". OvO is like running pairwise mini debates and then voting for winner.

✗ Watch out

OvO can be expensive when classes are many. OvR is usually simpler and fast, but sensitive to class imbalance across "rest" group.

✓ Key takeaway

Multiclass with logistic regression is usually handled through OvR or OvO wrappers. Choose based on number of classes, compute budget, and data balance.

12 Module Summary and Exam Checklist

- Logistic regression predicts class probability using sigmoid.
- Decision boundary at threshold 0.5 corresponds to linear score $z = 0$.
- Cross-entropy is the right loss for probabilistic classification.
- Evaluate with confusion matrix, precision, recall, F1, ROC, AUC.
- Handle class imbalance explicitly.
- Extend binary model to multiclass with OvR or OvO.

✓ Key takeaway

If you can explain sigmoid, cross-entropy, gradient update, confusion matrix metrics, ROC construction, and OvR vs OvO, you are exam-ready for this module.